A Deployment and Execution Framework for MoE Inference on 3D CIM Systems

***Abstract*—The high storage density of three-dimensional compute-in-memory (3D CIM) enables all experts of mixture-of-experts (MoE) models to reside on-chip, eliminating off-chip memory accesses for weight reads. Existing CIM compilation and simulation frameworks adopt different hardware representations and execution models, complicating MoE deployment and performance evaluation across 3D CIM architectures. Weight mapping and dataflow scheduling under dynamic routing must also balance decode latency against prefill throughput. We propose a deployment and execution framework for MoE inference that establishes a unified hardware abstraction and cost model for 3D CIM, connecting weight deployment, inference execution, and performance analysis. The framework optimizes expert replicas, spatial layouts, and execution locations through mapping and scheduling algorithms. Across three MoE models, adding approximately 20% more expert replicas reduces decode latency and prefill batch latency by 15.18%–18.46% and 3.86%–12.61%, respectively, compared with a single-replica deployment balanced by the full workload.**

***Index Terms*—3D compute-in-memory, mixture-of-experts, weight mapping, dataflow scheduling, performance modeling**

I. Introduction

Mixture-of-experts (MoE) models activate a small subset of expert feed-forward networks (FFNs) per token [1], yet inference requires large memory capacity to store all experts' weights and high bandwidth to read those of selected experts. Three-dimensional compute-in-memory (3D CIM) combines stacked weight-storage layers with in-situ matrix–vector multiplication, allowing all expert weights to be stored on-chip when capacity suffices [2]–[4]. Across a mesh of compute nodes, resident weights shift bottlenecks from off-chip weight reads to per-node computation and inter-node activation transfers.

Each node executes one resident operator at a time, so co-locating simultaneously activated experts on one node serializes their computation, whereas placing them on different nodes trades parallel execution against activation distribution and aggregation transfers. Within finite 3D node capacity, offline deployment shapes these paths through expert spatial layout across nodes and weight replication, which stores additional resident instances (replicas) of selected experts' weights on other nodes.

The same resident deployment must serve both decode and prefill under dynamic routing. Layouts or replica allocations that shorten decode latency can concentrate prefill work on heavily used nodes, whereas balancing aggregate prefill load can lengthen decode execution paths. Because all layers share each node's 3D storage capacity, the mapper must weigh spatial layout and additional replicas together to balance decode latency against prefill throughput.

Existing CIM compilation and simulation frameworks adopt different hardware representations and execution models across target arrays [2], [5]–[8]. Compilers for 2D arrays schedule static operator graphs, whereas 3D MoE studies evaluate single-replica deployments on specific memory stacks. Comparing candidate layouts and replica allocations across 3D CIM architectures requires a common hardware model that supplies 3D storage footprints and operator execution times to token-routed execution simulation.

We present a deployment and execution framework for MoE inference on 3D CIM systems. It models dense and expert FFN chains. Using a unified hardware model, the framework coordinates offline mapping and online scheduling to optimize one shared deployment and evaluate decode and prefill latencies alongside node and link loads. Our contributions are:

- We establish a unified hardware abstraction and cost model for 3D CIM that captures 3D weight storage footprints and operator execution times across computation and readout mechanisms.
- We propose a deployment and execution framework for MoE inference on 3D CIM that connects 3D capacity-constrained deployment, token-routed cross-layer execution simulation, and bottleneck profiling.
- We design weight mapping and dataflow scheduling algorithms that alternate replica allocation and spatial layout search offline, while jointly planning replica binding and result aggregation online for decode and prefill.

II. Background and Motivation

A. 3D CIM and MoE

CIM stores matrix weights in memory arrays and computes matrix–vector products where the weights reside. In analog implementations, input signals drive the programmed cells; accumulated currents or charges encode the weighted sums, which are digitized by the readout circuits. 3D CIM extends this organization with vertically stacked weight-storage layers [2]–[4]. The additional layers increase weight capacity per unit area and can share input and readout circuits, serializing co-resident operators within a node.

TABLE I
SCOPE OF CIM COMPILATION AND MOE DEPLOYMENT

Work	3D capacity	Routing data	Weight replicas	Replica selection	D/P balance
PIMCOMP [5]	–	–	✓	–	–
CIM-MLC [6]	–	–	✓	–	–
CIMFlow [7]	–	–	✓	–	–
3D AIMC [2]	✓	Token	–	–	–
OPTIMEX [8]	✓	Profile	–	–	–
This work	✓	Token	✓	✓	✓

3D capacity: stacked-array storage; routing data: per-token expert selections (Token) or activation statistics (Profile); weight replicas: static replication; replica selection: runtime dispatch among resident replicas; D/P balance: decode/prefill optimization on a shared deployment. ✓: explicit scope; –: outside scope.

An MoE layer replaces a dense FFN with expert FFNs. A gating network selects top-k experts per token; their outputs are combined using gating weights [1]. Some models include shared experts active on every token [9]. Prefill processes prompt tokens, whereas decode generates subsequent tokens autoregressively.

B. Related Work and Motivation

CIM compilers [5]–[7] model 2D crossbar or digital macro arrays. On 3D stacks, 3D AIMC [2] maps matrix chunks to analog tile tiers with fixed MVM and per-bit transfer latencies, whereas OPTIMEX [8] assigns expert projections to SRAM/ReRAM tiers using crossbar counts and vertical tier distances. Comparing 3D CIM designs across stacked geometries and readout circuits requires a unified footprint and operator-timing model.

Table I compares deployment and execution scope. 2D compilers pair weight replication with deterministic dataflows for static graphs [5]–[7]. Under MoE routing, 3D AIMC [2] schedules a token-wise operation graph on single-replica tiles without interconnect contention, while profile-guided methods optimize tier placement [8] or GPU-cluster affinity [10] offline. Our framework links 3D capacity-constrained mapping, contention-aware cross-layer simulation, and decode/prefill profiling on a shared deployment.

Co-activated experts on one node execute serially; separating them across nodes can lengthen activation transfers. Offline mapping combines spatial rearrangement, which relocates expert weights across nodes according to routing traces, with weight replication, which uses shared 3D capacity for additional resident instances (replicas) of selected experts' weights.

A fixed deployment still leaves coupled decode choices: nearest-replica dispatch ignores co-resident active compute, while a fixed aggregation node can lengthen branches. For a token at N0 activating E_1, E_2, E_3 in a 2×2 region of a 4×4 mesh (Fig. 1(a)), nearest-replica binding with node-index tie-breaking serializes E_1, E_2 on N1 and gathers outputs at N0, the root of the reduction tree (reduction root), in 6τ (Fig. 1(b)). Jointly binding E_2 to N4's E_2' and selecting N5 (E_3's node) as reduction root runs all three experts in parallel in 4τ (Fig. 1(c)), making N5 the next FFN's input source.

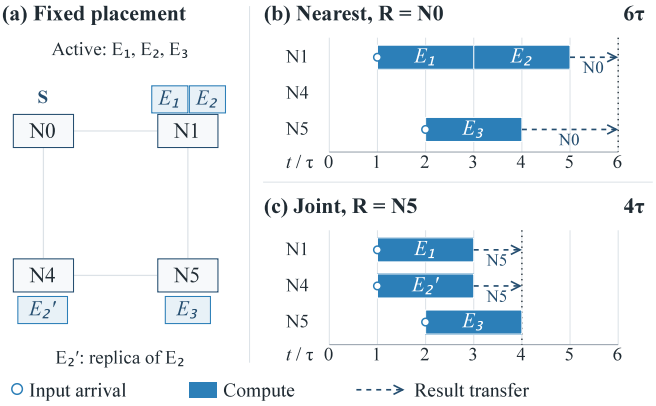


Fig. 1. Runtime scheduling on one fixed expert placement (2τ per expert, τ per hop, zero local summation time). S denotes input source; R denotes reduction root and next FFN source.

TABLE II
UNIT-CUBE AND MACRO GEOMETRY OF 3D-CIM DESIGNS

Design	Readout	Unit-Cube	Native macro
NAND PIM [3]	Planar	$128 \times 512 \times 128$	$16384 \times 65536 \times 128$
3D AIMC [2]	Planar	$512 \times 512 \times 256$	$8192 \times 8192 \times 256$
3D-FPIM [4]	Vertical	$128 \times 128 \times 64$	$512 \times 768 \times 64$
Sharma [8]	Planar	$128 \times 128 \times 16$	$12800 \times 13312 \times 16$
NASiC [11]	Planar	$128 \times w_u \times D_u$	$1024 \times 131072 \times 128$

Rows × columns × layers; logical node sizes are configured separately. Sharma: area-model geometry. NASiC: 128-input case, encoding-dependent w_u, and $D_u = 128 - D_{\mathrm{CAM}}$ after CAM reservation; plane dimensions follow SLs × BLs × WLs.

During prefill, per-expert round-robin splits each expert's requests across replicas yet ignores co-resident experts, which can overload shared nodes. The scheduler therefore selects replicas using each node's cumulative assigned execution time. Section III defines the 3D footprints, operator execution times, and transfer timings.

III. HARDWARE ABSTRACTION AND COST MODEL

The hardware model supplies 3D storage footprints, operator execution times, and external activation payloads to the mapper and simulator.

A. Architecture Model

A Unit-Cube stores D_u weight layers and performs an $h_u \times w_u$ local matrix–vector product, setting input-slice and output-partition granularity (Fig. 2(c)). A Sub-Cube groups units with activation buffers, input distribution, and result combination (Fig. 2(b)). Its extent $H \times W \times D_s$ counts logical rows, columns, and weight layers. A node adds control and a router; $G_H \times G_W$ nodes form the mesh.

Configurations specify geometry, activation width, and operator/network timing. Array parallelism and slice pipelining operate within an operator, whereas shared Sub-Cube circuits serialize co-resident operators on a node. Table II compares Unit-Cube and native macro geometries for five designs. Jang

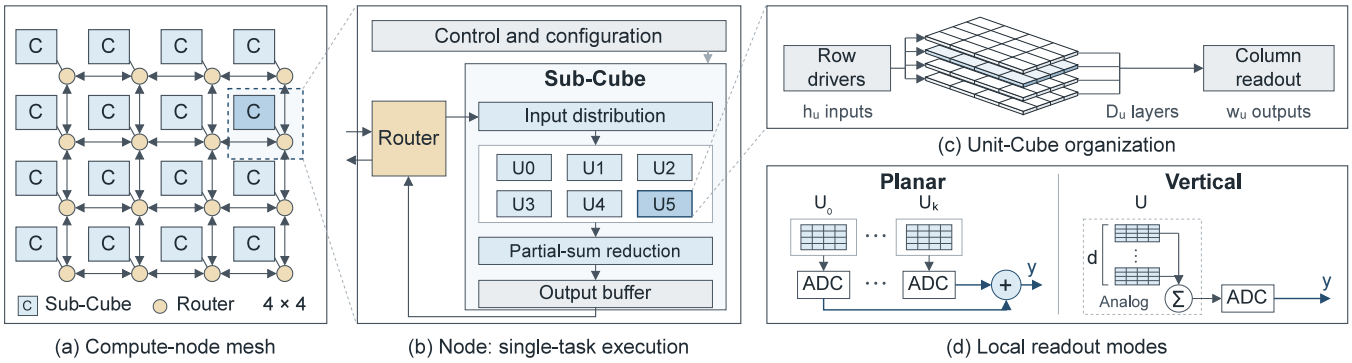


Fig. 2. Hardware hierarchy and local readout. (a) Compute nodes with Sub-Cubes (C) and routers. (b) Node resources surrounding a Sub-Cube. (c) A Unit-Cube with D_u weight layers and shared input and readout circuits. (d) Planar digitizes local results before digital reduction; Vertical accumulates d slices before conversion. Blue links carry digital results.

et al.'s NAND PIM and 3D-FPIM calibrate operator timing [3], [4].

B. Weight-Cube Representation

Each expert contains fused gate/up and down projections. For an operator with logical input/output lengths K_o, M_o, the stored column count is $C_w = 2M_o$ for fused gate/up and $C_w = M_o$ for down. The compiler produces a logical Weight-Cube descriptor:

$$c = \big((H_c, W_c, D_c),\, T_c,\, V_c^{\text{in}},\, V_c^{\text{out}}\big). \tag{1}$$

The compiler divides K_o inputs into $\lceil K_o/h_u \rceil$ slices. Planar maps slices within one weight layer ($d = 1$); Vertical stacks groups of d slices across d weight layers. After choosing d for the full operator (Section III-C), the compiler partitions input groups and stored columns into F_H and F_W sections to fit the node plane, padding to equal sizes. The resulting $F_c = F_H F_W$ sections each occupy $H_c \times W_c$ in the plane and d layers in depth. Consecutive depth ranges give $D_c = F_c d$.

The in-plane dimensions H_c, W_c align to h_u, w_u. Nonnegative placement origins (x_c, y_c, z_c) must satisfy $x_c + H_c \le H$, $y_c + W_c \le W$, and $z_c + D_c \le D_s$, with pairwise non-overlapping block extents. The mapper uses these conditions for placement feasibility.

The operator execution time T_c covers input loading, array computation, conversion, partial-sum combination, and output writes within one node. Section IV accounts for mesh transfers and contention for node and link resources. Across sections, the operator retains K_o inputs and M_o outputs; fused gate/up emits one gated M_o-element vector. With activation width b_{act}, $V_c^{\text{in}} = V(K_o, b_{\text{act}})$ and $V_c^{\text{out}} = V(M_o, b_{\text{act}})$, where $V(n, b) = \lceil nb/8 \rceil$ bytes.

C. Operator Execution Time

A section has logical lengths $K = \lceil K_o/F_H \rceil$ and $M = \lceil M_o/F_W \rceil$. Let $S = \lceil K/h_u \rceil$ and $Q = \lceil M/w_u \rceil$. Times are in nanoseconds and bandwidths in bytes per nanosecond.

Planar. The input path issues S slices at interval t_{issue}; the last result arrives after $t_{\text{mvm}} + (S-1)t_{\text{issue}}$. A fitted tail adds binary-tree reduction over $\lceil \log_2 S \rceil$ levels and writes across Q output partitions:

$$\begin{aligned} t_{\text{sec}}^{\text{P}}(K, M) &= t_{\text{mvm}} + (S-1)t_{\text{issue}} \\ &\quad + \text{round}(\tau_{\text{red}} \lceil \log_2 S \rceil) + \text{round}(\tau_{\text{out}} Q). \end{aligned} \tag{2}$$

The coefficients $\tau_{\text{red}}, \tau_{\text{out}}$ are fitted times per reduction level and output partition. Each tail term rounds separately to integer nanoseconds; input bandwidth B_{in} sets $t_{\text{issue}} = \lceil V(h_u, b_{\text{act}})/B_{\text{in}} \rceil$.

Vertical. The S slices form $P = \lceil S/d \rceil$ groups padded to depth d. Groups operate in parallel; each performs d sequential stack steps and one final conversion. The model serializes input loading, these array operations, $P-1$ digital vector additions, and output writing. With $T_{\text{in}}(K) = \lceil V(K, b_{\text{act}})/B_{\text{in}} \rceil$ and $T_{\text{out}}(M) = \lceil V(M, b_{\text{act}})/B_{\text{out}} \rceil$, section time is

$$\begin{aligned} t_{\text{sec}}^{\text{V}}(K, M, d) &= T_{\text{in}}(K) + t_0 + d\, t_{\text{step}} \\ &\quad + (P-1)t_{\text{add}} + T_{\text{out}}(M). \end{aligned} \tag{3}$$

The affine stack term combines setup, integration, switching, and conversion. Increasing d lengthens this sequence but can reduce P, trading analog time against digital combination.

The compiler chooses d using full input length K_o, minimizing the depth-dependent terms of (3) over $1 \le d \le \min(\lceil K_o/h_u \rceil, D_u)$. It then forms aligned sections and computes $T_c = F_c t_{\text{sec}}$. The expert execution time T_e sums its gate/up and down block times.

For Planar, $t_{\text{mvm}} = 2000$ ns and $t_{\text{issue}} = 64$ ns. Jang et al. report a simulated 8270-ns 7168×7168 MVM under hierarchy mapping [3]. With $S = 56, Q = 14$, the input–compute span is 5520 ns; equally splitting the 2750-ns fitted tail gives $\tau_{\text{red}} = 1375/6$ ns and $\tau_{\text{out}} = 1375/14$ ns.

For Vertical, 3D-FPIM's one setup, d integrations, $d-1$ switches, and final conversion contribute $105 + 32d + 170(d-1) + 256$ ns [4], giving $t_0 = 191$ ns and $t_{\text{step}} = 202$ ns. The evaluated profile sets $t_{\text{add}} = 128$ ns per digital vector addition.

D. Network Transfer Time

The expert's external input and output vectors cross mesh links; its gate/up intermediate vector remains local. For pay-

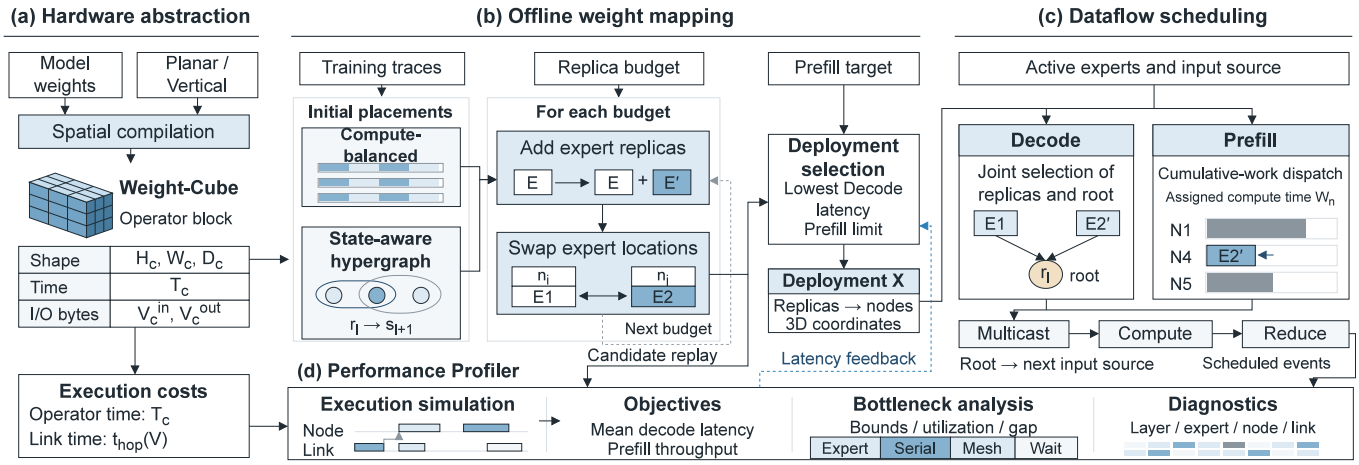


Fig. 3. Deployment and execution framework. Dashed arrows show budget progression and latency feedback.

load V, a directed link with bandwidth B_{mesh} provides packet serialization time

$$t_{\text{hop}}(V) = \left\lceil \frac{V}{B_{\text{mesh}}} \right\rceil. \tag{4}$$

A hop starts when its payload is ready and the directed link is free; opposite directions are independent. The evaluated policies use shortest-path multicast and reduction trees. A shared multicast edge carries one transfer; nodes combine available expert outputs before forwarding, with zero modeled inter-expert arithmetic time. The simulator in Section IV advances transfers using task dependencies and node/link availability.

IV. Inference Framework

A. Overview

Spatial compilation supplies Weight-Cubes alongside operator execution and per-hop transfer times (Fig. 3(a)) to the mapper, which uses historical expert traces to construct a shared deployment (Fig. 3(b)). The runtime scheduler binds active experts and selects roots (Fig. 3(c)); the simulator and profiler return latency feedback (Fig. 3(d)).

B. Framework Design

1) Weight Deployment: A replica is a resident instance of an expert's weights, including the initial instance. The compiler co-locates its gate/up and down Weight-Cubes on one node. A single packing routine verifies capacity and assigns block coordinates under Section III's constraints. Deployment X exposes expert e's resident-node set $\mathcal{R}_e$.

2) Inference Execution: Binding a maps each active expert e to $a_e \in \mathcal{R}_e$, forming $\mathcal{N}_a = \{a_e\}$. For source s and root r, let h count shortest-path hops and $t_{\text{in}}, t_{\text{out}}$ denote per-hop transfer times from (4). The planner scores the longest uncontended branch:

$$L(a,r) = \max_{n \in \mathcal{N}_a} \Big[h(s,n) t_{\text{in}} + \sum_{e:a_e=n} T_e + h(n,r) t_{\text{out}} \Big]. \tag{5}$$

While $L(a,r)$ scores uncontended branches, the simulator accounts for link contention. Equal-size expert input/output vectors and $T_e > 0$ make multicast reach a shared directed link before reduction traffic; the simulator adds multicast link-release times to reduction dependencies, waits for local and child results, and propagates roots and completion times across layers to obtain J_D. For prefill, node/link calendars advance concurrent tokens; computation priority is token index, operator stage (down before gate/up), then ready time, yielding batch makespan J_P.

3) Performance Profiler: The profiler sums timing contributions across all modeled FFN layers:

$$J_D = T_{\text{expert}} + T_{\text{node}} + T_{\text{mesh}} + T_{\text{wait}}. \tag{6}$$

T_{expert} combines dense FFN time with one T_e per MoE critical branch; remaining co-resident compute contributes to T_{node} (Serial in figures), and uncontended transfer to T_{mesh}. Residual T_{wait} captures delay from link availability and reduction dependencies. For prefill, W_C, W_L are maximum accumulated node-compute and directed-link times, and L_{dep} the longest per-token compute chain:

$$B_{\text{exec}} = \max(W_C, W_L, L_{\text{dep}}) \le J_P. \tag{7}$$

The profiler reports the bound gap $J_P - B_{\text{exec}}$ alongside node and link utilization.

C. Algorithms

1) Weight Mapping: Two single-replica initializers start independent budget tracks. The compute-balanced initializer counts fixed dense work, then places experts by descending activation frequency on least-loaded feasible nodes. The state-aware hypergraph initializer labels expert vertices with resident nodes; each sampled training trace q supplies one co-activation hyperedge per MoE layer.

A layer's root becomes the next input source, coupling placement across layers. For trace q entering from s, let $C_\ell(q,s;X) = T_{\text{node},\ell} + T_{\text{mesh},\ell}$ denote critical-branch serialization and transfer time, and $r_\ell(q,s;X)$ the decode planner's

chosen root. The mapper scores placement with the remaining sum of C_ℓ terms, $V_\ell(q, s; X)$:

$$V_\ell(q, s; X) = C_\ell(q, s; X) + V_{\ell+1}\big(q, r_\ell(q, s; X); X\big), \tag{8}$$

with zero suffix after the last layer. Before each forward Fiduccia–Mattheyses (FM) sweep [12], the mapper builds the suffix table backward for every sampled trace and every mesh source node. Candidate moves update affected C_ℓ terms and roots, then query cached $V_{\ell+1}$ at the new roots. Capacity-constrained FM passes retain the best move prefix and propagate the resulting roots as next-layer input sources.

Each track continues from its previous-budget layout, first greedily adding replicas at feasible nodes to reduce projected peak node compute work, splitting each expert's expected work equally among replicas. At fixed replica counts and node occupancy, same-layer swap search then minimizes mean decode latency while keeping prefill batch times within 2% of that track's replica-addition candidate at every evaluated batch size.

Validation compares the two tracks' candidates at each budget. At each prefill batch size, the faster candidate sets the reference. The mapper selects minimum mean decode latency within 2% maximum relative prefill overhead; if neither qualifies, it uses the smallest feasible maximum overhead.

2) Dataflow Scheduling: For decode, the scheduler jointly minimizes (5) for the current layer over resident-node bindings and all mesh nodes as roots. Lower bounds on L for partial bindings prune the search.

The prefill scheduler spreads concurrent work across replicas. Each window starts with cumulative assigned compute time $W_n = 0$; the scheduler binds experts sequentially and immediately updates the selected node's work [13]:

$$a_e = \arg\min_{n \in \mathcal{R}_e} (W_n + T_e), \qquad W_{a_e} \leftarrow W_{a_e} + T_e. \tag{9}$$

The next assignment uses the updated W_n. Among equal-work candidates, the scheduler prefers minimum input transfer time, then node index. After binding, it minimizes reduction-tree edge count, with maximum return hop count, total return hop count, and node index as successive tie-breakers.

V. Evaluation

A. Experimental Setup

We evaluate dense and expert FFNs in DeepSeek-V3, DeepSeek-V2-Lite, and Qwen2-57B-A14B [14]–[16]. DeepSeek-V3 uses seed-0 topic-correlated synthetic routes. DeepSeek-V2-Lite uses PrefetchMLP decode [17] and MoE-Beyond prefill routes [18]; Qwen2 uses HD-MoE sequences [13]. Optimization, validation, and test use disjoint 4,096-token blocks; hypergraph fitting uses 1,024 training traces. Prefill uses 32/128/512-token windows (T32/T128/T512), with one validation window and eight test windows per size.

On a 4×4 mesh (8192×16384 node planes), a $1.5\times$ FFN weight-volume target gives stack depths $D_s = 460, 11, 39$ for the three models, respectively. Planar uses 8-bit activations and 2-GB/s local I/O; Vertical uses 5 bits and 1 GB/s, following Section III. DeepSeek-V3 runs at 0.5/2/8 GB/s mesh bandwidth within each readout configuration at fixed expert execution times; the other models use 2 GB/s. Additional replica budgets cover 0–20% of single-replica MoE units, counting Qwen2's shared FFN as eight routed-width units. Section IV's simulator computes latencies. Each configuration uses a compute-balanced single-replica baseline under the same runtime policies. Figs. 4–5 report decode speed (J_D^{base}/J_D) and prefill throughput (J_P^{base}/J_P); text reports reductions in mean latency.

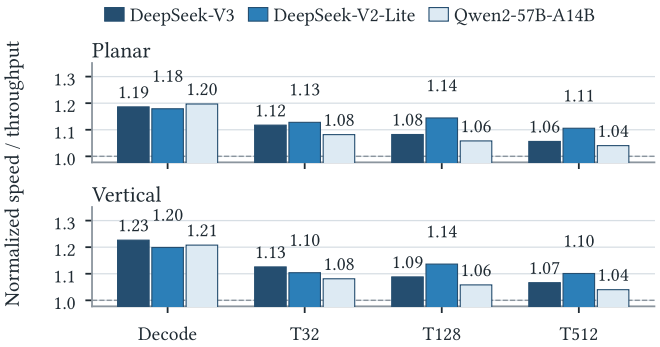


Fig. 4. Cross-model test performance at 2 GB/s and ≈ 20% additional replicas (compute-balanced single-replica baseline = 1).

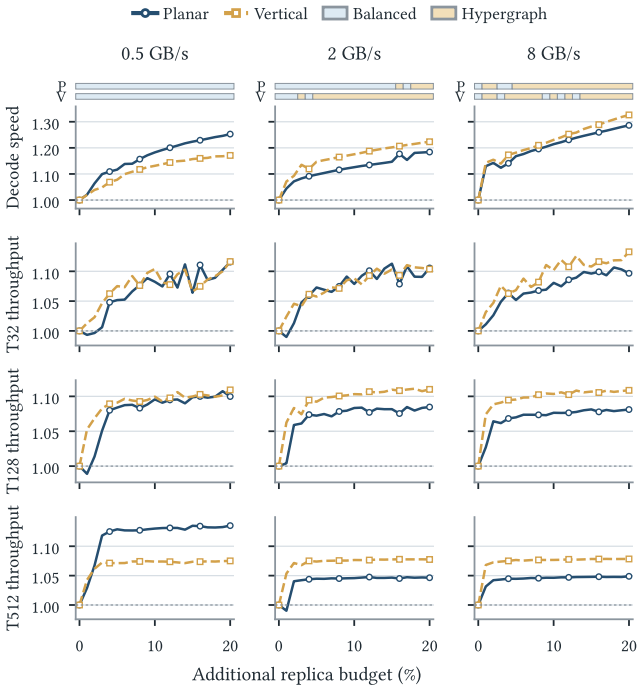


Fig. 5. DeepSeek-V3 validation sweep (compute-balanced single-replica = 1). P/V strips show selected tracks for Planar/Vertical.

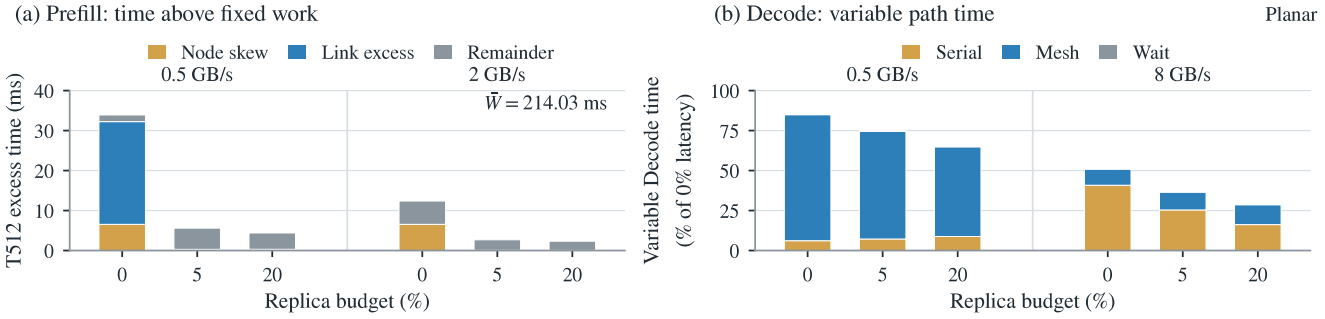


Fig. 6. Planar validation latency across replica budgets. (a) T512 time above mean node compute time $\bar{W}$. (b) Decode components excluding fixed T_{expert}, normalized by the same-bandwidth 0%-budget total latency.

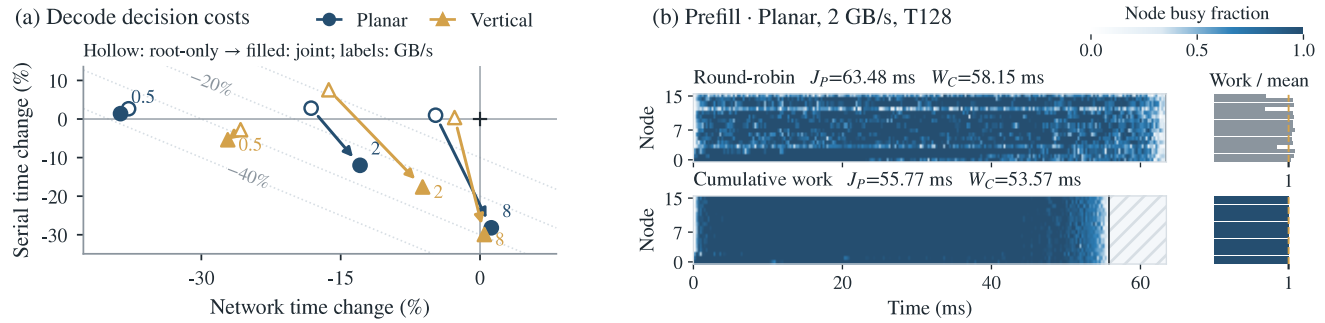


Fig. 7. Scheduling on test routes with frozen ≈ 20% deployments. (a) Network ($T_{\text{mesh}} + T_{\text{wait}}$) and serialization changes normalized by nearest/source-root J_D; diagonals show total change. (b) Node activity in the first test window; hatching marks time after completion.

B. Overall Performance

At 2 GB/s, the selected deployments with ≈ 20% additional replicas reduce decode latency by 15.18–18.46% and prefill batch latency by 3.86–12.61% against their respective baselines (Fig. 4).

C. Prefill Work Distribution

For fixed DeepSeek-V3 validation routes and hardware, replication and spatial rearrangement preserve the FFN computations. Window completion time obeys $J_P \geq W_C \geq \bar{W}$, where $\bar{W}$ is the average accumulated node compute time. The T512 excess $J_P - \bar{W}$ in Fig. 6(a) comprises node skew $W_C - \bar{W}$, link excess $\max(W_L - W_C, 0)$, and execution residual. The 20%-budget Planar 2-GB/s T512 latency is 1.09% above $\bar{W}$. Planar peak work approaches $\bar{W}$ by ≈ 5% budget; across all six configurations, this budget achieves 91.65–97.93% of T512's 20%-budget latency reduction (Fig. 5). The zero-budget Planar T512 case at 0.5 GB/s is link-bound.

The 20%-budget Planar 2-GB/s execution residuals span 2.06–2.29 ms across T32/T128/T512. As node compute work grows with window size, these similar absolute residuals occupy a smaller fraction of latency.

With 20% deployments frozen, per-expert round-robin balances request counts yet can accumulate different experts' requests on one shared node. Under identical root-selection and tree-routing rules, cumulative-work dispatch lowers Planar T128 peak work from 58.15 to 53.57 ms (Fig. 7(b)). Across six DeepSeek-V3 configurations and three window sizes, mean test batch latency falls by 4.30–14.62% relative to round-robin.

D. Decode Execution Choices

Decode continues to improve over 10–20% budget (Fig. 5) as additional replicas and spatial rearrangement provide more execution choices. Expert computation remains fixed within each configuration (Fig. 6(b)). Across the Planar budget sweep, the 0.5-GB/s deployments reduce mesh transfer time while accepting more node serialization; at 8 GB/s, they reduce serialization while allowing more transfer time.

In the Planar 0.5-GB/s validation case, a 1%-budget hypergraph candidate lowers decode latency by 21.50% but raises T512 latency by 17.21% against the compute-balanced single-replica baseline. The selector retains the compute-balanced track under the prefill guard.

On frozen 20% deployments, root-only planning (nearest binding, movable root) recovers most of joint planning's 0.5-GB/s benefit (Fig. 7(a)). The test baseline uses nearest binding with the input source as root. Moving the root changes the next layer's source and can redirect later bindings. At 8 GB/s on Planar, root-only reduces latency by 3.80%; joint planning reduces it by 26.98%, chiefly through less serialization among co-activated experts.

REFERENCES

[1] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022. [Online]. Available: https://jmlr.org/papers/v23/21-0998.html

[2] J. Buechel, A. Vasilopoulos, W. A. Simon, I. Boybat, H. Tsai, G. W. Burr, H. Castro, B. Filipiak, M. L. Gallo, A. Rahimi, V. Narayanan, and A. Sebastian, "Efficient scaling of large language models with mixture of experts and 3D analog in-memory computing," *Nature Computational Science*, vol. 5, pp. 13–26, 2025.

[3] Y. Jang, S. Hwang, H. Lee, S. Jung, D. Lee, W. Shim, and J. Kung, "Dissecting and re-architecting 3D NAND Flash PIM arrays for efficient single-batch token generation in LLMs," in *2025 IEEE 43rd International Conference on Computer Design (ICCD)*, 2025, pp. 237–244.

[4] H. Lee, M. Kim, D. Min, J. Kim, J. Back, H. Yoo, J.-H. Lee, and J. Kim, "3D-FPIM: An extreme energy-efficient DNN acceleration system using 3D NAND Flash-based in-situ PIM unit," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 1359–1376.

[5] X. Sun, X. Wang, W. Li, L. Wang, Y. Han, and X. Chen, "PIMCOMP: A universal compilation framework for crossbar-based PIM DNN accelerators," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.

[6] S. Qu, S. Zhao, B. Li, Y. He, X. Cai, L. Zhang, and Y. Wang, "CIM-MLC: A multi-level compilation stack for computing-in-memory accelerators," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 185–200.

[7] Y. Qi, J. Yang, Y. Wang, Y. Wang, D. Wang, L. Tang, C. Duan, X. He, and W. Zhao, "CIMFlow: An integrated framework for systematic design and evaluation of digital CIM architectures," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–7.

[8] P. Sharma, A. R. Bommana, F. Firouzi, and K. Chakrabarty, "Taming sparse giants: Deploying mixture-of-experts on 3D heterogeneous compute-in-memory systems," in *2025 IEEE 43rd International Conference on Computer Design (ICCD)*, 2025, pp. 285–288.

[9] D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024. [Online]. Available: https://arxiv.org/abs/2401.06066

[10] J. Yao, Q. Anthony, A. Shafi, H. Subramoni, and D. K. Panda, "Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference," *arXiv preprint arXiv:2401.08383*, 2024. [Online]. Available: https://arxiv.org/abs/2401.08383

[11] W. Xu, M. Li, S. Zhong, T. Luo, D. Zhao, L. Liang, Z. Wang, Q. Huang, Y. Cai, and R. Huang, "NASiC: 3D NAND-based CAM-selected multibit CIM architecture for efficient on-device mixture-of-experts LLM inference," *arXiv preprint arXiv:2605.23294*, 2026. [Online]. Available: https://arxiv.org/abs/2605.23294

[12] C. M. Fiduccia and R. M. Mattheyses, "A linear-time heuristic for improving network partitions," in *19th Design Automation Conference*, 1982, pp. 175–181.

[13] H. Huang, S. Zhong, Z. Zhang, S. Li, D. Niu, H. Zheng, R. Wang, and M. Li, "HD-MoE: Hybrid and dynamic parallelism for mixture-of-expert LLMs with 3D near-memory processing," in *2025 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2025, pp. 1–9.

[14] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024. [Online]. Available: https://arxiv.org/abs/2412.19437

[15] ——, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024. [Online]. Available: https://arxiv.org/abs/2405.04434

[16] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou *et al.*, "Qwen2 technical report," *arXiv preprint arXiv:2407.10671*, 2024. [Online]. Available: https://arxiv.org/abs/2407.10671

[17] bubblebird, "PrefetchMLP," Hugging Face Dataset, 2026. [Online]. Available: https://huggingface.co/datasets/bubblebird/PrefetchMLP

[18] N. Gavhane, A. Mehrotra, R. Chawla, and P. Proenca, "MoE-Beyond: Learning-based expert activation prediction on edge devices," *arXiv preprint arXiv:2508.17137*, 2025. [Online]. Available: https://arxiv.org/abs/2508.17137